

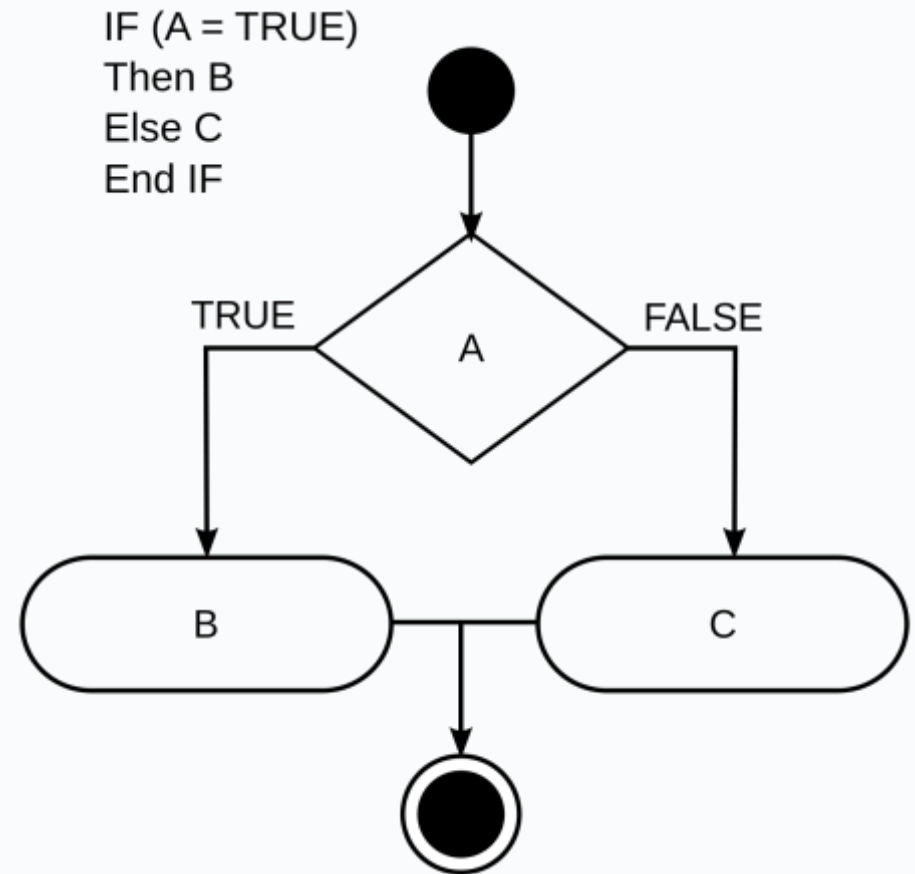
# **Conditional Expressions and Subqueries**

**Dr Mariana Rocha**  
**School of Computer Science**

CPMU 2007 Databases 1, Lecture 8

# Conditional Expressions

Provide the use of IF-THEN-ELSE logic within a SQL statement



# CASE expression

- The CASE expression goes through a list of conditions, returning a value once a condition is met (like an if-then-else statement).
- If a true condition is met, the query stops reading the data and return the result.
- If no conditions are true, it returns the value in the ELSE clause.
- If there is no ELSE part and no conditions are true, it returns NULL.

```
--syntax
SELECT attribute,
CASE
    WHEN condition1 THEN result1
    WHEN condition2 THEN result2
    WHEN conditionN THEN resultN
    ELSE result
END
FROM tablename;
```

# CASE expression

- PostgreSQL lets you use CASE almost anywhere:
  - SELECT list
  - WHERE clause
  - ORDER BY clause

# Simple CASE

- A simple CASE compares one column to a fixed set of values.
- It's often used to translate coded attributes (such as positions or rating levels) into clearer categories.

```
SELECT
  p.player_name,
  pa.position,
  CASE pa.position
    WHEN 'GK' THEN 'Goalkeeper'
    WHEN 'CB' THEN 'Defender'
    WHEN 'RB' THEN 'Defender'
    WHEN 'LB' THEN 'Defender'
    WHEN 'CM' THEN 'Midfielder'
    WHEN 'CDM' THEN 'Midfielder'
    WHEN 'CAM' THEN 'Attacker'
    WHEN 'LW' THEN 'Attacker'
    WHEN 'RW' THEN 'Attacker'
    WHEN 'ST' THEN 'Attacker'
    ELSE 'Other / utility'
  END "Position category"
FROM players p
JOIN player_attributes pa USING
(player_id)
LIMIT 15;
```

# Searched CASE

- A searched CASE is more flexible.
- It can use different conditions for each WHEN clause.
- It will evaluate a set of Boolean expressions (TRUE or FALSE) and return the result for the first TRUE condition

```
SELECT
pc.player_id,
p.player_name,
pc.wage,
CASE
WHEN pc.wage >= 50000 THEN 'Tier 1:
Elite'
WHEN pc.wage >= 10000 THEN 'Tier 2:
First Team'
WHEN pc.wage >= 2000 THEN 'Tier 3:
Rotation'
WHEN pc.wage IS NULL THEN 'No
contract / unknown'
ELSE 'Tier 4: Youth / Low cost'
END "Salary band"
FROM player_contracts pc
JOIN players p USING (player_id);
```

# CASE and WHERE

- The CASE statement in the WHERE clause is evaluated for each row, and it returns a value based on the conditions.
- The returned value can be used to apply different WHERE conditions based on the result.

# CASE and WHERE

- Suppose you want retrieve players following specific conditions about their wage.
- For big clubs (like Real Madrid or Chelsea), you only want players earning more than 50,000/week.
- For smaller clubs, you want players earning more than 5,000/week.
- You can write that conditional logic using CASE inside the WHERE clause.



# CASE and WHERE

```
SELECT
  p.player_name,
  c.club_name,
  pc.wage
FROM players p
JOIN player_contracts pc USING (player_id)
JOIN clubs c USING (club_id)
WHERE
  pc.wage >
CASE
  WHEN c.club_name IN ('Real Madrid', 'Chelsea') THEN
    50000
  ELSE 5000
END
ORDER BY c.club_name, pc.wage DESC;
```

# CASE and Maths operations

- We can also use the CASE expression combined with mathematics operations to modify the value shown in the output.
- Suppose we want to simulate an updated wage budget for next season:
  - Increase wages by 10% for players earning less than €10,000/week.
  - Increase wages by 5% for players earning between €10,000 and €50,000/week.
  - No increase for players above €50,000/week.

# CASE and Maths operations

```
SELECT
p.player_name,
c.club_name,
pc.wage,
CASE
WHEN pc.wage < 10000 THEN pc.wage * 1.10
WHEN pc.wage BETWEEN 10000 AND 50000 THEN pc.wage * 1.05
ELSE pc.wage
END "Projected wage"
FROM players p
JOIN player_contracts pc USING (player_id)
JOIN clubs c USING (club_id)
WHERE pc.wage IS NOT NULL
ORDER BY c.club_name, pc.wage DESC;
```

# CASE and SQL functions

- You're preparing a financial report that groups clubs into "High Budget", "Medium Budget", or "Low Budget" tiers based on their average player wage.

```
SELECT
c.club_name,
ROUND(AVG(pc.wage)) "Average wage",
CASE
WHEN AVG(pc.wage) >= 40000 THEN 'High Budget Club'
WHEN AVG(pc.wage) BETWEEN 10000 AND 39999 THEN 'Medium
Budget Club'
WHEN AVG(pc.wage) < 10000 THEN 'Low Budget Club'
ELSE 'No Data'
END "Budget category"
FROM clubs c
JOIN player_contracts pc USING (club_id)
WHERE pc.wage IS NOT NULL
GROUP BY c.club_name
ORDER BY AVG(pc.wage) DESC;
```

# CASE and LIKE

We can also establish the case expression considering string values by combining it with the LIKE operator:

```
SELECT
  p.player_name,
  CASE
    WHEN p.player_name LIKE '%Junior%' THEN 'Brazilian-style name'
    WHEN p.player_name LIKE '%inho' THEN 'Brazilian-style name'
    ELSE 'Other pattern'
  END AS name_pattern
FROM players p
ORDER BY 2 ASC;
```

# CASE and LIKE

We can also establish the case expression considering string values by combining it with the LIKE operator:

```
SELECT
  p.player_name,
  CASE
    WHEN p.player_name LIKE '%Junior%' THEN 'Brazilian-style name'
    WHEN p.player_name LIKE '%inho' THEN 'Brazilian-style name'
    ELSE 'Other pattern'
  END AS name_pattern
FROM players p
ORDER BY 2 ASC;
```

# CASE in ORDER BY

- Suppose we want to list all players, but sort them by their playing role priority instead of alphabetically. For instance:
  - Attackers first
  - Then Midfielders
  - Then Defenders
  - Finally Goalkeepers

# CASE in ORDER BY

```
SELECT
p.player_name,
pa.position,
pa.overall
FROM players p
JOIN player_attributes pa USING (player_id)
ORDER BY
CASE
WHEN pa.position IN ('ST', 'LW', 'RW', 'CAM') THEN 1 -- Attackers
WHEN pa.position IN ('CM', 'CDM', 'LM', 'RM') THEN 2 -- Midfielders
WHEN pa.position IN ('CB', 'LB', 'RB', 'RWB', 'LWB') THEN 3 -- Defenders
WHEN pa.position = 'GK' THEN 4 -- Goalkeepers
ELSE 5 -- Unknown / other
END;
```



# Subquery in SQL

- A subquery is a query nested inside a bigger query
- The subquery can be nested inside a SELECT, INSERT, UPDATE, or DELETE statement, or inside another subquery

# Subquery in WHERE clause

2)

```
SELECT attribute_name FROM table_name WHERE  
condition (SELECT attribute FROM table_name  
WHERE condition);
```

1)

- 1) The subquery (inner query) executes once before the main query (outer query).
- 2) The result of the subquery is used by the main query.



# Single-row subqueries

- Return only one row from the sub-query
- Use single-row comparison operators

| Operator | Meaning                  |
|----------|--------------------------|
| =        | Equal to                 |
| >        | Greater than             |
| >=       | Greater than or equal to |
| <        | Less than                |
| <=       | Less than or equal to    |
| <>       | Not equal to             |



# Single-row subqueries

- Suppose we want to know the players who are earning more than the average wage.
- We can use a subquery:

```
SELECT
p.player_name,
pc.wage
FROM players p
JOIN player_contracts pc USING (player_id)
WHERE pc.wage >
(SELECT AVG(wage) FROM player_contracts);
```

# Combining single-row subqueries

- We can also combine multiple single-row subqueries to write more complex conditions.
- Suppose we want to list all players who earn more than Chelsea's top earner and are worth more than the average Real Madrid player.

```
SELECT p.player_name, pc.wage
FROM players p
JOIN player_contracts pc USING (player_id)
WHERE pc.wage > (
  SELECT MAX(wage) FROM player_contracts WHERE club_id =
  'CL137' -- Chelsea
) AND pc.value > (
  SELECT AVG(value) FROM player_contracts WHERE club_id =
  'CL473' -- Real Madrid
);
```

# EXISTS

- The EXISTS keyword is typically used to check for the existence of rows in the subquery, and it returns true if the subquery returns one or more rows.
- SELECT 1 is a shorthand used in EXISTS subqueries to check for the existence of at least one row that satisfies the specified conditions in the subquery.
- For example, let's list all clubs that currently have at least one player earning over 50k/week

```
SELECT c.club_id, c.club_name
FROM clubs c
WHERE EXISTS (
  SELECT 1
  FROM player_contracts pc
  WHERE pc.club_id = c.club_id
  AND pc.wage >= 50000
);
```

# Multiple-row subqueries

- Return more than one row from the sub-query
- Use multiple-row comparison operators

| Operator | Meaning                                                                                                          |
|----------|------------------------------------------------------------------------------------------------------------------|
| IN       | Equal to any member in the list                                                                                  |
| NOT IN   | Unequal to any member in the list                                                                                |
| ANY/SOME | Compare a value to each value returned by the subquery (commonly used with comparison operators such as > and <) |
| ALL      | Compare value to every value returned by the subquery                                                            |



# IN Operator

- The IN operator is used to check if a value matches any value in a subquery.
- Let's find players who are contracted to clubs that have at least one high-earning player (someone earning €50,000 or more).

```
SELECT
p.player_name,
c.club_name,
pc.wage
FROM players p
JOIN player_contracts pc USING (player_id)
JOIN clubs c USING (club_id)
WHERE pc.club_id IN (
SELECT DISTINCT club_id
FROM player_contracts
WHERE wage >= 50000
);
```



# NOT IN Operator

- The NOT IN operator is used to check if a value does not match any value in a subquery.
- Suppose we want to know what players currently DO NOT have a contract.

```
SELECT
p.player_name
FROM players p
WHERE p.player_id NOT IN (
SELECT player_id
FROM player_contracts
);
```

# Using the ANY operator

- The ANY operator compares a value to each value returned by a subquery and returns a row if meets the condition for any of them.
- Suppose we want players whose wage is higher than any player at Chelsea (meaning higher than at least one Chelsea player).

```
SELECT
p.player_name,
pc.wage
FROM players p
JOIN player_contracts pc USING (player_id)
WHERE pc.wage > ANY (
  SELECT wage
  FROM player_contracts
  WHERE club_id = 'CL137' -- Chelsea
);
```

# Using the ALL operator

- The ALL operator compares a value to every value returned by a subquery and returns a row only if it meets the condition for all of them
- Suppose we want to find players whose wage is higher than every player at Chelsea (so, higher than Chelsea's top earner).

```
SELECT
  p.player_name,
  pc.wage
FROM players p
JOIN player_contracts pc USING
  (player_id)
WHERE pc.wage > ALL (
  SELECT wage
  FROM player_contracts
  WHERE club_id = 'CL137' -- Chelsea
);
```

# DELETE

- Removes all rows which satisfy the condition
- Does not free the memory space
- It is slower than truncate since it operates in each row individually
- If no condition is given then ALL rows are deleted - BE

CAREFUL

# DELETE

```
--DELETE
```

```
DELETE FROM player_contracts  
WHERE contract_valid_until < 2020;
```

```
SELECT * FROM player_contracts WHERE  
contract_valid_until < 2020;
```

# Truncate

- Removes all of the table data without saving any rollback information
- It frees up the space
- Faster than delete because it deallocates the entire data at once
- It cannot be used with a condition
- Must disable foreign key constraints before truncating table

# Truncate

- Removes all of the table data without saving any rollback information
- It frees up the space
- Faster than delete because it deallocates the entire data at once
- It cannot be used with a condition
- Must disable foreign key constraints before truncating table

```
--TRUNCATE
```

```
TRUNCATE TABLE player_attributes;
```

# Update

- All rows where the condition is true have the columns set to the given values
- If no condition is given all rows are changed so BE CAREFUL
- Values are constants or can be computed from columns

```
--UPDATE  
UPDATE player_contracts  
SET wage = wage * 1.10  
WHERE wage < 10000;
```



# Update with subquery

- Let's update all players who have NULL wages and set their wage to the average wage of all players.

```
UPDATE player_contracts
SET wage = (
SELECT ROUND(AVG(wage))
FROM player_contracts
WHERE wage IS NOT NULL
)
WHERE wage IS NULL;
```